

진행 상황

이번 회의에서는 전날 구현한 MVP를 바탕으로 WakeBuddy의 실제 서비스 기능을 확장하고, Firebase 연동 및 최종 화면 구조를 반영한 구현 방향을 정리하였다. 기존에는 임시 데이터를 사용하여 화면과 핵심 기능 흐름을 확인하는 데 집중했다면, 이번 단계에서는 실제 사용자 인증, 데이터 저장, 친구 요청, 친구 알람 생성 기능까지 연결하는 것을 목표로 하였다.

전날에는 MVP 구현을 목표로 Expo 기반 React Native 프로젝트를 생성하고, 기본 폴더 구조와 임시 데이터 기반 기능을 구현하였다.

구분	구현 내용
개발 환경	Expo 프로젝트 생성 및 Expo Go 실행 확인
프로젝트 구조	src/components, src/services, src/models, src/firebase, src/constants, src/utils 폴더 생성
데이터 모델	User, Alarm, Friendship 타입 정의
인증 기능	임시 데이터 기반 회원가입, 로그인, 로그아웃 흐름 구현
알람 기능	임시 데이터 기반 내 알람 조회, 생성, 수정, 삭제, 활성화/비활성화 구현
친구 기능	임시 데이터 기반 친구 목록 조회, 친구 추가, 친구 삭제 구현
화면 구현	Home, Login, SignUp, MyAlarm, AlarmForm, FriendList 화면 구현
GitHub 관리	기능 단위 커밋 및 브랜치 분리 진행

전날 구현은 실제 서버나 데이터베이스와 완전히 연결된 상태는 아니었으며, 앱 화면과 주요 기능 흐름이 동작하는지 확인하는 MVP 단계로 진행하였다.

이번 구현 목표

이번 구현 단계에서는 MVP에서 사용하던 임시 데이터를 제거하고, 실제 서비스에서 사용할 수 있도록 Firebase Authentication, Cloud Firestore, Expo Notifications 연동을 진행하기로 하였다. 또한 요구사항이 추가됨에 따라 친구 추가 방식을 단순 추가 방식이 아니라 친구 요청, 수락, 거절, 취소 흐름으로 변경하였다.

구분	구현 목표
인증 연동	Firebase Authentication 기반 회원가입, 로그인, 로그아웃 구현
사용자 저장	회원가입 시 Firestore users 컬렉션에 사용자 정보 저장
알람 저장	Firestore alarms 컬렉션에 알람 데이터 저장
친구 관계 저장	Firestore friendships 컬렉션에 친구 관계 저장
친구 요청	friendRequests 컬렉션을 사용하여 요청, 수락, 거절, 취소 구현
친구 알람	친구에게 알람을 생성하고, 생성자와 대상자를 구분하여 저장
화면 구조 개선	로그인 전 화면과 로그인 후 화면을 분리
설정 화면	설정 탭에 계정 정보와 로그아웃 버튼 배치
알림 기능	Expo Notifications 기반 알림 권한 요청, 알람 예약, 알람 취소 구조 준비

변경된 최종 화면 구조

전날 MVP에서는 Home 화면에서 로그인, 회원가입, 내 알람, 친구 관리로 바로 이동하는 구조였으나, 최종 구현에서는 로그인 여부에 따라 화면 구조를 분리하였다.

상태	화면 구조
비로그인 상태	Main 화면에서 로그인, 회원가입 버튼만 표시
로그인 상태	AppHome 화면에서 알람, 친구, 설정 탭 표시
알람 탭	내 알람 조회, 생성, 수정, 삭제, 활성화/비활성화
친구 탭	친구 목록, 친구 추가, 신청 현황, 수신함 관리
설정 탭	사용자 계정 정보 확인, 로그아웃

로그아웃 기능은 별도의 설정 화면에 배치하여 사용자가 로그인 후 계정 상태를 관리할 수 있도록 하였다.

브랜치별 구현 내용

이번 단계에서는 기능 충돌을 줄이기 위해 브랜치를 역할별로 나누어 작업하였다.

브랜치	담당 영역	주요 작업
feat/auth	인증 및 친구 관계	Firebase 인증, 사용자 저장, 친구 관계, 친구 요청, 권한 확인
feat/alarm	알람 데이터 및 알림	알람 모델 확장, Firestore 알람 CRUD, 알림 예약 서비스 구조
feat/screens	화면 UI 및 화면 연결	로그인 전/후 화면, 알람 화면, 친구 화면, 설정 화면, 친구 알람 생성 연결

feat/auth 구현 내용

feat/auth 브랜치에서는 임시 사용자 데이터를 제거하고 Firebase Authentication 기반 인증 기능을 구현하였다. 회원가입 시 Firebase Authentication에 계정을 생성하고, 동시에 Firestore users 컬렉션에 사용자 프로필을 저장하도록 구성하였다.

파일	구현 내용
src/firebase/firebaseConfig.ts	Firebase 앱, Authentication, Firestore 초기화
src/models/User.ts	사용자 데이터 타입 정의
src/models/Friendship.ts	친구 관계 데이터 타입 정의
src/models/FriendRequest.ts	친구 요청 데이터 타입 정의
src/services/AuthService.ts	회원가입, 로그인, 로그아웃, 로그인 상태 감지 구현
src/services/FriendService.ts	친구 목록 조회, 친구 삭제, 친구 여부 확인, 친구 관계 생성 구현
src/services/FriendRequestService.ts	친구 요청 보내기, 취소, 수락, 거절 구현
src/services/PermissionService.ts	친구 알람 생성 권한 확인 구현

친구 추가 기능은 기존의 즉시 친구 등록 방식에서 친구 요청 방식으로 변경하였다. 사용자가 친구 이메일을 입력하면 friendRequests 컬렉션에 pending 상태의 요청이 저장되고, 상대방은 수신함에서 해당 요청을 수락하거나 거절할 수 있도록 하였다. 친구 요청을 수락하면 friendships 컬렉션에 친구 관계 문서가 생성된다.

feat/alarm 구현 내용

feat/alarm 브랜치에서는 기존 임시 알람 데이터를 제거하고 Firestore 기반 알람 저장 기능을 구현하였다. 기존에는 time 필드 하나만 사용했으나, 실제 알람 앱에 가깝게 날짜, 시간, 반복 여부, 반복 요일을 분리하여 저장하도록 모델을 확장하였다.

파일	구현 내용
src/models/Alarm.ts	알람 데이터 타입 확장
src/services/AlarmService.ts	Firestore 기반 알람 조회, 생성, 수정, 삭제, 활성화/비활성화 구현
src/services/NotificationService.ts	알림 권한 요청, 알림 예약, 알림 취소, 알림 재예약 구조 구현
src/utils/alarmSchedule.ts	반복 알람 날짜 계산 보조 함수 구현

Alarm 모델에는 ownerId와 creatorId를 두어 내 알람과 친구 알람을 구분할 수 있도록 하였다.

구분	ownerId	creatorId
내 알람	내 uid	내 uid
친구가 나에게 만든 알람	내 uid	친구 uid
내가 친구에게 만든 알람	친구 uid	내 uid

이를 통해 사용자는 자신의 알람뿐 아니라 친구가 자신에게 설정한 알람도 조회할 수 있고, 자신이 친구에게 생성한 알람도 별도로 조회할 수 있도록 하였다.

feat/screens 구현 내용

feat/screens 브랜치에서는 최종 화면 구조를 구현하고, 각 화면에서 서비스 함수를 호출하도록 연결하였다. 기존 MVP 화면 일부는 최종 구조에 맞게 제거하거나 대체하였다.

파일	구현 내용
App.tsx	로그인 상태 감지, 전체 화면 전환 관리
MainScreen.tsx	비로그인 상태에서 로그인, 회원가입 버튼만 표시
LoginScreen.tsx	로그인 화면
SignUpScreen.tsx	회원가입 화면
AppHomeScreen.tsx	로그인 후 알람, 친구, 설정 탭 관리
AlarmListScreen.tsx	내 알람과 내가 만든 친구 알람 조회
AlarmFormScreen.tsx	알람 생성, 수정, 날짜/시간/반복/요일 선택
FriendTabScreen.tsx	친구 목록, 친구 추가, 신청 현황, 수신함 관리
SettingsScreen.tsx	계정 정보 표시 및 로그아웃

기존 MVP 파일 중 HomeScreen, MyAlarmScreen, FriendListScreen은 최종 화면 구조에서 MainScreen, AlarmListScreen, FriendTabScreen으로 대체하였다.

친구 알람 생성 기능 구현

이번 구현에서 추가된 주요 기능은 친구 알람 생성이다. 친구 알람은 별도의 모델을 새로 만들지 않고 기존 Alarm 모델의 ownerId와 creatorId를 활용하여 구현하였다.

친구 알람 생성 흐름은 다음과 같다.

단계	처리 내용
1	친구 탭에서 친구 목록 조회
2	친구 카드에서 알람 생성 버튼 클릭
3	AlarmFormScreen으로 이동
4	알람 제목, 날짜, 시간, 반복 여부, 요일 선택
5	PermissionService로 친구 알람 생성 권한 확인
6	AlarmService.createAlarm 호출
7	Firestore alarms 컬렉션에 저장

(ownerId = 친구 uid, creatorId = 현재 로그인한 사용자 uid)

다음 회의 안건

주요 내용
알람 기능 버그 수정
iOS / Android 알람 동작 차이 확인
최종 테스트 및 앱 배포 여부 논의
전체 기능 안정화 및 UI 점검